



Programmation Orientée Objets

(II 1)

Les classes en c++



Les Classes en C++

- Déclaration et définition d'une classe
- Constructeur et Destructeur
- Surcharge des opérateurs
- Données statiques
- Généricité

Rappels

- Une classe est une implémentation d'un type abstrait de données,
- Une classe est la généralisation de la notion de type défini par l'utilisateur, dans lequel se trouvent associées des données « Membre données » (ou **attributs**) et des fonctions « Fonctions membre » ou **méthodes**.
- En POO pure, les données sont « encapsulées » => leur accès ne peut se faire que par le biais des méthodes.

Déclaration d'une classe

(Fichier Point.h)

```
/* ----- Déclaration de la classe point ----- */
```

```
class Point
```

```
{ /* déclaration des membres privés, inaccessible en dehors de la classe */
```

```
private : /* facultatif */
```

```
int x ;
```

```
int y ;
```

```
/* déclaration des membres publics, accessible en dehors de la classe */
```

```
public :
```

```
void initialise (int, int) ;
```

```
void deplace (int, int) ;
```

```
void affiche () ;
```

- ➔ public et private peuvent apparaître plusieurs fois.
- ➔ Si aucun, tous les membres de la classe seront privés (rarement utile)
- ➔ En P.O.O pure, les données membres sont encapsulées (private) (en C++ vous avez le choix ➔ pas recommandé)

Déclaration, définition et utilisation d'une classe

Fichier Point.cpp

```
/* ----- Définition des fonctions membres de la classe point ----- */  
void Point::initialise (int abs, int ord) //Définition d'initialize qui appartient à la classe Point  
{ x = abs ; y = ord ;  
}  
void Point::deplace (int dx, int dy)  
{ x = x + dx ; y = y + dy ;  
}  
void Point::affiche ()  
{ cout << "Je suis en " << x << " " << y << "\n" ;  
}
```

Fichier main.cpp

```
/* ----- Utilisation de la classe point ----- */  
int main()  
{ Point a, b ; /* instantiation de la classe Point en deux objets a et b*/  
a.initialise (5, 2) ; a.affiche () ; // on applique les méthodes initialise et affiche sur l'objet a  
a.deplace (-2, 4) ; a.affiche () ; // on applique les méthodes deplace et affiche sur l'objet a  
b.initialise (1,-1) ; b.affiche () ; // on applique les méthodes initialise et affiche sur l'objet b
```

Exemple :

```
class ABCD {  
    int i,j;  
    public :  
        int f() { return i;}  
};
```

Déclaration de la classe ABCD

...

```
void main()  
{
```

```
    ABCD a;
```

instantiation de la classe ABCD
en un objet a

```
    int i = a.f();
```

on applique la méthode f
à l'objet a

```
}
```

Exemple 2

Déclaration de la classe point

```
class point
{ // déclaration des membres privés
private :
  int x ;
  int y ;

  // déclaration des membres publics
public :
  void initialise (int, int) ;
  void deplace (int, int) ;
  void affiche () ;
};
```

```
void main()
{
  point p;

  p.initialise(-2,5);
}
```

Instantiation de la classe point
en un objet p

On applique la méthode initialise
à l'objet p

Protection/visibilité

```
class Point
{
private :
    int x ;
    int y ;
Public:
    void initialise (int, int)
    void deplace (int, int) ;
    void affiche () ;
};
```

```
int main()
{
    Point a;

    a.initialise(5,2);
    a.deplace (6, 3);
    a.affiche();

    a.x = 5; //illégal!!!
}
```

illégal

Protection/visibilité

```
class Point
{ // déclaration des membres privés
private :
  int x ;
  int y ;

  // déclaration des membres publics
public :
  void initialise (int, int) ;
  void affiche () ;

Private :
  void deplace (int, int) ;
};
```

```
int main()
{
  Point p;
  p.initialise(5,2);
  p.deplace(3,6);
}
```

illégal



Affectation d'objets

```
class Point
{
int x ;
public:
int y ;
};
```

```
int main()
{
Point a,b;

b=a; //recopie des valeurs des membres x et y de a
      // dans les membres x et y de b, publics ou privés

}
```

→ Contrairement aux structures, il n'est pas toujours possible de remplacer cette instruction par :

```
b.x = a.x ; // illégale car x est un membre privé.
b.y = a.y ;
```

→ L'affectation $a = b$ est toujours légale, quel que soit le statut (public ou privé) des membres données

→ Elle ne viole pas le principe d'encapsulation, dans la mesure où les données privées de b (les copies de celles de a , après affectation) restent toujours inaccessibles de manière directe.

Surcharge des méthodes

```
class A {  
    public :  
        int f() { return i;}  
        void f(int val ) { i=val;}  
    private :  
        int i;  
};
```

```
int main()  
{  
    A a;  
  
    a.f(10);  
  
    cout<<a.f()<<"\n";  
}
```



Affiche 10

Modificateurs (setters) et Accesseurs (getters)

- Une méthode marquée **const** est un **accesseur** : elle ne doit **pas modifier l'état de l'objet**.

```
class A {  
    public :  
        int f()  
        {  
            //modifie l'attribut i,  
            //f n'est pas un accesseur  
            i = 0;  
            return i;}  
        void f(int val)  
        {  
            //modifie l'objet i  
            // f est un modificateur  
            i=val;}  
    private :  
        int i;  
};
```

```
...  
int f()  const  
{  
    i = 0;  
    return i;  
}  
...
```

- ➔ Une fonction censée **retourner une valeur** (int f()) ne devrait pas modifier l'objet.
- ➔ Bonne pratique utiliser **const** après la signature

Modificateurs (setters) et Accesseurs (getters)

```
class A {  
public:  
    int getI() const {  
        return i;  
    }  
  
    void setI(int val) {  
        i = val;  
    }  
  
private:  
    int i;  
};
```

- Le mot-clé **const**, placé après la signature d'une méthode:
 - indique que celle-ci est un accesseur
 - et garantit qu'elle ne modifie pas l'état de l'objet
 - toute tentative de modification entraînant un diagnostic de compilation.

Définition des méthodes

- Deux façons différentes pour définir les méthodes d'une classe:
 - Dans la classe
 - En dehors de la classe

```
class A {  
    public :  
        int f();  
        void f(int val) { i=val; }  
    private :  
        int i;  
};
```

Méthode **inline** par défaut

```
int A::f()  
{  
    return i;  
}
```

Spécification de contexte ::

Exemple: Expliquer?

```
char c; ← ::c
class X      // X::
{
public:
    char c; ← X::c
    class Y      // X::Y::
    {
    public:
        void f(char e) { X t; ::c = t.X::c = c = e; }
    private:
        char c; ← X::Y::c
    };
};
```

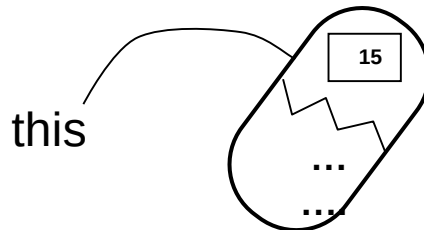
Le pointeur "this"

```
class A {  
    public :  
        int f();  
        void f(int);  
    private :  
        int i;  
};
```

Lors de la définition de la méthode void f(int):
Relever l'ambiguïté des noms: i paramètre de la
fonction et i attribut de la classe!

```
void f(int i)  
{  
    return this->i = i;  
}
```

- ➔ **this** peut être interprété comme un identifiant de l'objet courant
- ➔ Il fait toujours référence à l'instance pour laquelle la méthode est invoqué.



Le pointeur "this"

```
class A {  
    public :  
        int f();  
        void f(int);  
    private :  
        int i;  
};
```

```
void f(int i)  
{  
    return this->i = i;  
}
```

```
Int main () {  
    A a;  
    a.f(10);  
    ...  
}
```

- ➔ **this** pointe vers a
- ➔ **this->i = i** devient: a.i = 10;

Les opérateurs de création et de destruction des objets

```
class A {  
    public :  
        int f();  
        ...  
};
```

```
char * text;  
/* création d'un tableau dynamique de 80 char sur heap */  
text = new char[80];  
...  
delete[] text; //destruction du tableau
```

```
A * a;
```

(A*) malloc(sizeof(A));

```
a = new A; //créer objet a sur heap  
int i = a->f(); //appeler f sur cet objet  
...  
delete a; //détruire l'objet a
```

free a;

Le constructeur et le destructeur

→ L'initialisation des attributs a été faite par le modifieur `void f(int);`



Risque d'oubli !!

■ Constructeur

- Une **fonction membre** portant le **même nom** que sa classe se nomme un **constructeur**,
- Un constructeur peut avoir des arguments mais ne renvoie pas de valeur (**aucune indication de type même pas `void`**).
- Le constructeur **est appelé implicitement** tout juste après l'allocation de l'espace mémoire destiné à l'objet,

■ Destructeur

- Une **fonction membre** portant le **même nom** que sa classe, précédé du symbole **tilde** (**~**), se nomme **destructeur**,
- Un destructeur **ne peut pas comporter d'arguments** et il **ne renvoie pas de valeur** (aucune indication de type ne doit être prévue).
- Le destructeur **est appelé implicitement avant la libération de l'espace mémoire associé à l'objet**,

Constructeur & destructeur

```
class A {  
public :  
A();  
A(int);  
...  
~A();  
...  
};
```

Un constructeur de A

Un constructeur surdéfini de A

Le destructeur

```
Int main()  
{  
A a;  
A * ptr;  
cout<<"\n"<<a.f();  
...  
ptr = new A(1);  
cout<<ptr->f();  
...  
delete ptr;  
}
```

Objet automatique

Le constructeur de A est exécuté automatiquement par A().

Le destructeur est appelé à la sortie de la fonction ou du bloc de déclaration.

Le destructeur de A est exécuté

Constructeur/ Destructeur

```
class A {  
public :  
A();  
A(int);  
...  
~A();...  
};
```

Un constructeur de A

Un constructeur surdéfini de A

Le destructeur

```
int main()  
{  
A a;  
A * ptr;  
cout<<"\n"<<a.f();  
...  
ptr = new A(1);  
cout<<ptr->f();  
...  
delete ptr;  
}
```

Objet **dynamique**

Le constructeur A(int) est appelé

Réalise une allocation dynamique d'espace mémoire pour un élément de type A et affecte son adresse au pointeur ptr.

Le destructeur de A est exécuté

Constructeur & destructeur

```
class A {  
public :  
    A();  
    A(int);  
    ...  
    ~A();  
    ...  
};
```

Un constructeur de A

Un constructeur surdéfini de A

Le destructeur

```
int main()  
{
```

```
    A a;  
    A * ptr;  
    printf("%d \n", a.f());  
    ...  
    ptr = new A(1);  
    ...  
    delete ptr;  
}
```

Le constructeur de A est exécuté par A()

Le constructeur de A est exécuté par A(int)

Le destructeur de A est exécuté

Le destructeur de A est exécuté

Constructeur & Destructeur

- Les objets **automatiques** sont ceux créés par une **déclaration** :
 - **Dans une fonction:**
 - L'objet est créé lors de la rencontre de sa déclaration: A a;
 - Il est détruit à la fin de l'exécution de la fonction.
 - **Dans un bloc:**
 - l'objet est créé lors de la rencontre de sa déclaration. A a;
 - Il est détruit lors de la sortie du bloc.

Constructeur & Destructeur

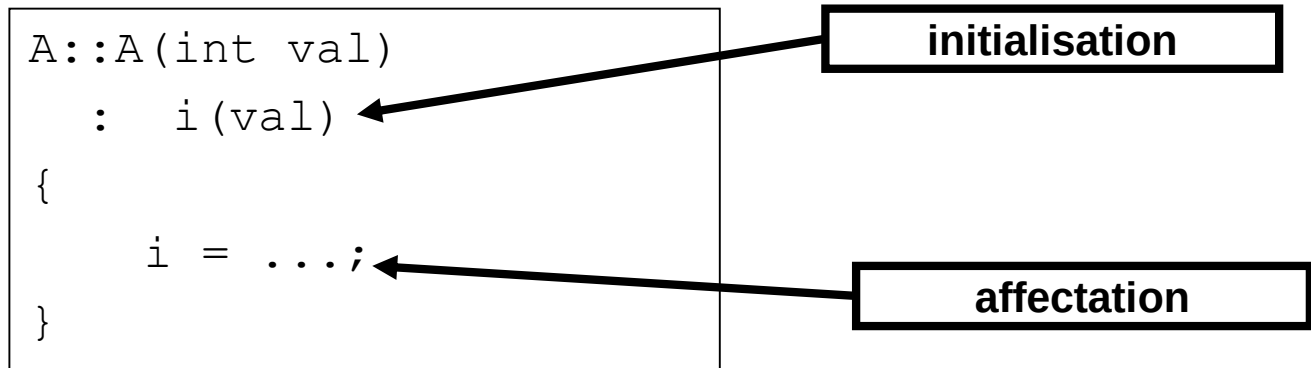
- Les objets **automatiques** sont ceux créés par une **déclaration** :
 - **Dans une fonction:**
 - L'objet est créé lors de la rencontre de sa déclaration: A a;
 - Il est détruit à la fin de l'exécution de la fonction.
 - **Dans un bloc:**
 - l'objet est créé lors de la rencontre de sa déclaration. A a;
 - Il est détruit lors de la sortie du bloc.

Constructeur & Destructeur

- Les objets **statiques** sont ceux créés par une **déclaration** située :
 - en dehors de toute fonction,
 - dans une fonction, mais assortie du qualificatif **static**.
static A a;
- Les objets statiques sont créés avant le début de l'exécution de la fonction main et détruits après la fin de son exécution.

Constructeur (...)

- Le corps du constructeur peut être précédé par **une section optionnelle** introduite par ":", spécifiquement réservée à **l'initialisation des attributs (membres données de la classe)**.
- Il est possible de changer la valeur des attributs dans le corps du constructeur
- Les attributs non initialisés dans cette section prendront une valeur par défaut dans le cas où une telle valeur existe



Remarque : on utilise "this" si le nom de l'attribut est identique au nom de l'argument

Constructeur par défaut

- Lorsqu'on **ne définit aucun** constructeur pour une classe, le compilateur se charge de générer automatiquement **un constructeur par défaut** (minimaliste = initialisation par défaut des attributs)

```
class A {  
    int i;  
public :  
    A() : i(10) { }  
};
```

```
class B {  
    int j;  
public :  
    B(int x) : j(x) { }  
};
```

```
class C {  
    ...  
    A objA;  
    B objB;  
};
```

```
C() :  
    objA(),  
    objB()  
{ }
```

auto-généré

?

Constructeur ayant 1 seul paramètre

- Déclaration d'un constructeur ayant un unique paramètre et l'utiliser pour faire la conversion.

```
class A {  
    int x;  
public:  
    A(int v) : x(v) {}  
};  
void f(A obj) {  
    // ...  
}
```

```
A a = 5;           // équivalent à : A a(5);  
A a(7);           // OK, conversion explicite  
A b = A(7);       // OK, conversion explicite  
a = 9;            // conversion A(9) + affectation a.x=9  
f(10);            //10 convertis implicitement en A(10)
```

- ➔ A(int) est un constructeur à un parameter
- ➔ Il définit une conversion implicite de int vers A

- ➔ Le compilateur transforme automatiquement 5 (int) en A(5) .
- ➔ C'est ce qu'on appelle **faire la conversion: Un constructeur de conversion**

Constructeur ayant 1 seul paramètre

- Déclaration d'un constructeur ayant un unique paramètre et l'utiliser pour faire la conversion.

```
class A {  
public:  
    A(int);  
};  
void f(A) {}  
void g() {  
    A a1 = 37;  
    A a2 = A(47);  
    A a3(57);  
    a1 = 67;  
    f(77);  
}
```

A a1 = 37;

Creation d'un objet A avec une valeur entière. Le constructeur est appelé "converting constructor".

Clonage des objets

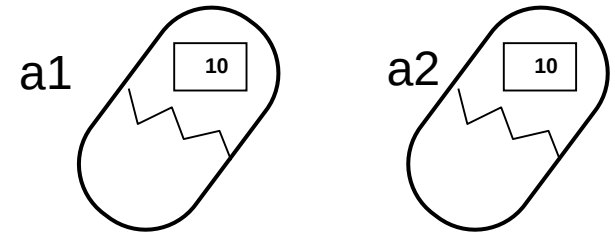
Constructeur de copie

Forme générale

```
<id_classe> (const <id_classe> & obj) : ...  
{  
}
```

```
A::A(const A & a)  
: i(a.i)  
{  
    i = ...;  
}
```

```
void main()  
{  
    A a1;  
    A a2(a1);  
}
```



Remarque : s'il n'est pas défini, le compilateur génère un de manière automatique
(initialisation membre à membre en invoquant le constructeur de copie de chacun d'eux)

```
Point (const Point& obj ){x=obj.x ;  
y=obj.y;}
```

Constructeur de recopie...

```
class Bidon
{
    double * adr ; // pointeur sur 1 élément

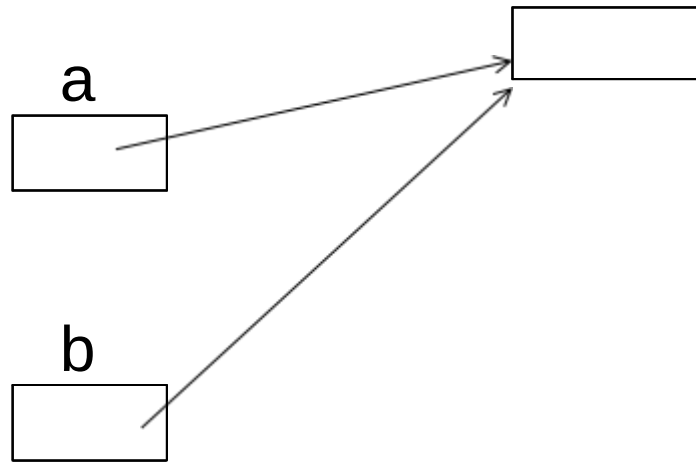
public :
    Bidon() // constructeur "usuel"
    { adr = new double;
      cout << "+ const. usuel - adr objet : " << this
        << " - adr : " << adr << "\n" ;
    }

    ~Bidon() // destructeur
    { cout << "- Destr. objet - adr objet : "
      << this << " - adr : " << adr << "\n" ;
      delete adr ;
    }
};
```

```
void fct (Bidon )
{ cout << "*** appel de fct ***\n" ;
}

main()
{
    Bidon a ;
    fct (a) ;
}
```

Constructeur de copie...



A la fin de l'exécution de la fonction fct, le destructeur `~Bidon` est appelé pour b, ce qui libère l'emplacement pointé par adr ;

A la fin de l'exécution de la fonction main, le destructeur est appelé pour a, ce qui libère... Le même emplacement.

Cette tentative constitue une erreur d'exécution dont les conséquences varient avec l'implémentation.

Constructeur de recopie...

```
class Bidon
{
double * adr ; // pointeur sur 1 élément

public :
Bidon() // constructeur "usuel"
{...}

Bidon(const Bidon & v) // constructeur de recopie
{
adr = new double ;
// création nouvel objet

*adr = *(v.adr);
// recopie des contenus des ptr

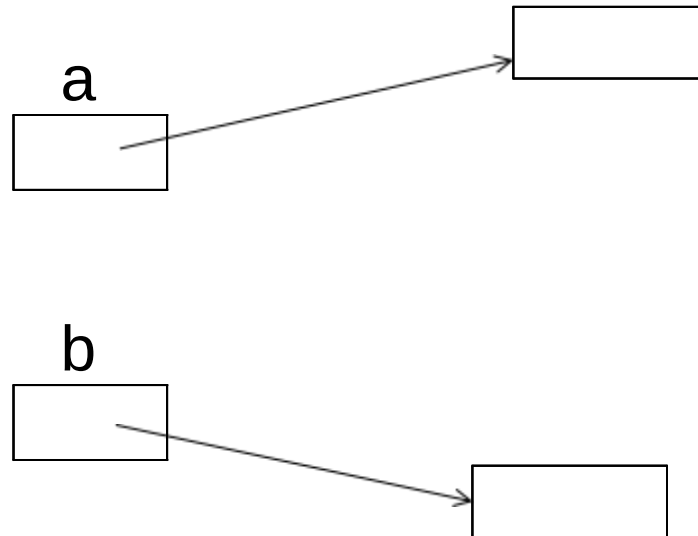
cout << "+ const. recopie - adr objet : " << this
<< " - adr : " << adr << "\n" ;
}

~Bidon() // destructeur
{ ....}
};
```

```
void fct (Bidon)
{ cout << "*** appel de fct ***\n" ;
}

main()
{
Bidon a ;
fct (a) ;
}
```

Constructeur de copie...



Chaque objet possédant son propre emplacement mémoire, les destructions successives se déroulent sans problème.